# Auditing Content Moderation AI for Bias, Adversarial Robustness & Safety

## 1. The Scenario

You are a Trust & Safety engineer at a mid-sized social media platform. Six months ago, your team deployed a BERT-based toxicity classifier to automatically moderate user-generated content. The classifier has been running in production ever since, flagging comments for removal or review.

Last week, your civil rights team escalated a serious complaint. Analysis of moderation decisions shows the classifier is flagging comments associated with Black identity at nearly twice the rate of otherwise identical comments in standard American English. This is not a hypothetical edge case. A 2019 Stanford NLP study documented exactly this pattern in classifiers trained on the Jigsaw dataset  the same dataset your classifier was trained on.

The platform's legal team is asking hard questions. The policy team wants answers. You have one week to:

- Understand exactly where the bias originates and how large it is
- Understand how adversarial actors could exploit the classifier's weaknesses
- Apply mitigation techniques and measure whether they actually work
- Design a production-grade guardrail pipeline that is robust to both bias and attacks
- Deliver all of this as working, reproducible code

## 2. Dataset

### Which dataset and why

You will use one dataset for the entire assignment: the Jigsaw Unintended Bias in Toxicity Classification dataset, released by Google's Jigsaw team and hosted on Kaggle. This is a real dataset that was used to train production toxicity classifiers. It is the canonical benchmark for studying bias in content moderation AI, and nearly every academic paper on this topic uses it. Working with it is a direct job skill.

### How to download

The dataset is free but requires a Kaggle account and competition acceptance.

1. Create a free account at kaggle.com if you do not already have one.
2. Go to: kaggle.com/c/jigsaw-unintended-bias-in-toxicity-classification
3. Go to the Data tab and download only the files listed below. You do not need all 5 GB.

## Which files to use

The competition provides ten files. You need exactly two of them:

| File | Use | Why |
|---|---|---|
| **jigsaw-unintended-bias-train.csv** | Training + evaluation | Contains identity columns needed for the bias audit |
| **validation.csv** | Optional sanity check | Can be used to verify model behavior independently |

**Why not the other files?**

- **jigsaw-toxic-comment-train.csv** This is the older, simpler Jigsaw dataset from a previous competition. It has no identity annotation columns. You cannot do a bias audit with it. Ignore it.
- ***-seqlen128.csv files** These are pre-tokenized versions for BERT. You will tokenize the text yourself using HuggingFace's tokenizer, which is better practice and more flexible. Ignore them.
- **test.csv + test_labels.csv** These are non-English comments from the Kaggle competition test set. The assignment uses English data only. Ignore them.
- **sample_submission.csv** A Kaggle competition artifact. Ignore it entirely.

## Columns you will actually use

The file jigsaw-unintended-bias-train.csv contains approximately 1.8 million rows. The columns relevant to this assignment are:

| Column | Type | Description |
|---|---|---|
| **comment_text** | string | The raw comment text. This is your model input. |
| **toxic** | float 0–1 | The toxicity score assigned by human raters. Binarize at ≥ 0.5 to get your binary label. A score ≥ 0.5 means the comment was considered toxic by the majority of annotators. |
| **black** | float 0–1 | Soft probability score indicating the comment references or is associated with Black identity. Assigned by crowdworkers. You will use this to construct your high-black cohort for the bias audit. |
| **white** | float 0–1 | Soft probability score for White identity association. Used to construct the reference cohort. |
| **muslim, jewish, lgbtq** | float 0–1 | Additional identity columns. Optional: you may use these to extend your bias audit beyond the two primary cohorts. |

## Working subset sizes

The full file has ~1.8 million rows, which is impractical to train on without high-end GPUs. You will work on a stratified random sample. Stratify on the binary toxic label so that the class balance in your subset matches the full dataset.

- Training subset: 100,000 rows
- Evaluation subset: 20,000 rows (held out, never seen during training)
- Sampling strategy: stratified random sample on the binarized toxic column

The sampling and stratification code is your responsibility to write. This is intentional  correctly splitting data while preserving class balance and avoiding leakage is a fundamental skill.

# 3. Tools & Environment

You must use the following tools. No alternatives are accepted for the core libraries since these are the industry-standard tools for this problem class.

| Library | Purpose in this assignment |
|---|---|
| **transformers (HuggingFace)** | Fine-tuning DistilBERT, tokenization |
| **torch** | Backend for DistilBERT training |
| **scikit-learn** | Metrics, CalibratedClassifierCV, train/test splits |
| **fairlearn** | ThresholdOptimizer for post-processing fairness mitigation |
| **aif360** | Reweighing pre-processor, ClassificationMetric for audit metrics |
| **pandas** | Data loading, filtering, cohort construction |
| **matplotlib / seaborn** | All plots (ROC, confusion matrices, bar charts, Pareto curve) |

Google Colab (free tier) is sufficient for all parts if you use the GPU runtime. Training DistilBERT on 100,000 rows for 3 epochs takes approximately 25–35 minutes on a T4 GPU. You are free to use a local environment if you have access to a CUDA GPU.

# 4. Assignment Parts

## Part 1  Build the Baseline Classifier

**What, why, and how**

**What:** Fine-tune a pre-trained DistilBERT model on the Jigsaw dataset to classify comments as toxic or non-toxic.

**Why:** You need a working production-representative model before you can audit, attack, or fix it. DistilBERT is chosen specifically because it is the same family of model used in real content moderation systems, it is small enough to train in under 30 minutes on a free Colab GPU, and it produces calibrated probabilities rather than just binary labels  which matters for the guardrail design in Part 5.

**How:** Use HuggingFace's Trainer API for fine-tuning. This is the industry-standard way to fine-tune transformer models and avoids writing a custom training loop from scratch.

**Step-by-step tasks**

1. Load jigsaw-unintended-bias-train.csv and create your 100,000-row training subset and 20,000-row evaluation subset using stratified sampling on the binarized toxic label.
2. Binarize the toxic column: any row with toxic ≥ 0.5 becomes label 1, anything below becomes label 0. This is the standard convention for this dataset.
3. Initialize distilbert-base-uncased from HuggingFace and tokenize all comment_text with max_length=128 and truncation=True.
4. Fine-tune for 3 epochs using the Trainer API with standard cross-entropy loss. No custom loss function is required here.
5. On your evaluation subset, report: Accuracy, F1 (macro), AUC-ROC, and the confusion matrix.
6. Plot the ROC curve and the Precision-Recall curve.
7. Justify your operating threshold in a markdown cell. The default threshold is 0.5, but this may not be optimal. Show the F1 score at several thresholds (0.3, 0.4, 0.5, 0.6, 0.7) and state which you will use for the rest of the assignment and why.

**Key question to answer (in a markdown cell in your notebook)**

What does your chosen threshold imply about the platform's priorities? A lower threshold catches more toxic content but increases false positives, meaning innocent users get incorrectly flagged. A higher threshold reduces false positives but misses more genuine toxicity. There is no universally correct answer you must take a position and justify it.

**Deliverable:** part1.ipynb with all metrics printed and all plots rendered in output cells. Save your model checkpoint to disk (you will need it in later parts).

## Part 2 Bias Audit: Who Does the Model Hurt?

**What, why, and how**

**What:** Systematically measure whether your classifier treats comments associated with Black identity differently from comments associated with White identity, across multiple fairness metrics.

**Why:** A classifier that is "accurate on average" can still cause serious real-world harm if its errors are concentrated on specific demographic groups. This is the civil rights problem described in the scenario. Documenting the disparity precisely is the prerequisite for fixing it.

**How:** Use the identity columns in the Jigsaw dataset to construct two subgroup cohorts from your evaluation set. The black and white columns are soft scores (0–1) assigned by human annotators indicating how strongly a comment references or is associated with a given identity. They are your fairness-relevant attributes.

**Constructing the cohorts**

From your 20,000-row evaluation subset, filter as follows:

- **High-black cohort:** rows where black ≥ 0.5. These are comments that annotators associated with Black identity.
- **Reference cohort:** rows where black < 0.1 AND white ≥ 0.5. These are comments associated with White identity and not associated with Black identity.

These thresholds follow the methodology used in the academic literature on this dataset. Report how many rows fall into each cohort at the start of your notebook.

**Metrics to compute**

For each cohort separately, compute and report:

- True Positive Rate (TPR) of genuinely toxic comments, what fraction does the model catch?
- False Positive Rate (FPR) of non-toxic comments, what fraction does the model incorrectly flag?
- False Negative Rate (FNR) of genuinely toxic comments, what fraction does the model miss?
- Precision of flagged comments, what fraction are actually toxic?
- **Disparate Impact ratio:** FPR(high-black) / FPR(reference). A ratio above 1.0 means the model over-flags the high-black cohort.

Additionally, use aif360's ClassificationMetric to compute:

- **Statistical parity difference:** the difference in positive prediction rates between cohorts.
- **Equal opportunity difference:** the difference in TPR between cohorts.

**Visualizations required**

- A grouped bar chart showing TPR, FPR, and FNR side-by-side for the two cohorts
- A confusion matrix for each cohort separately (two confusion matrices total)

**Key question to answer (in a markdown cell in your notebook)**

Which metric shows the largest disparity? Does the model predominantly over-flag the high-black cohort (high FPR), under-flag it (high FNR), or both? What are the real-world consequences of each error direction for the platform and for users in each cohort?

**Deliverable:** part2.ipynb with cohort sizes printed, all metrics in a summary table, the grouped bar chart, and both confusion matrices rendered in output cells.

## Part 3 Adversarial Attacks: Breaking the Classifier

**What, why, and how**

**What:** Implement two adversarial attacks a character-level evasion attack and a label-flipping poisoning attack and measure how badly each degrades your classifier.

**Why:** Content moderation systems are adversarial by design. Bad actors are not passive they actively probe classifiers for weaknesses and adjust their behavior accordingly. Before any system goes into production, its attack surface must be documented. This is standard practice at every major platform's trust and safety team.

**How:** You will implement both attacks from scratch in Python, without using an adversarial ML library. This forces you to understand the mechanics rather than just calling a function.

**Attack 1 Character-level evasion**

The goal of an evasion attack is to modify a toxic comment just enough that the classifier no longer detects it, while keeping the content recognizable to a human reader.

Write a function perturb(text) that applies all three of the following transformations to the input string:

- **Zero-width space insertion:** Insert the Unicode character U+200B (zero-width space) between every 2–3 characters of "toxic-looking" words. This is invisible to human readers but splits the subword tokens that the model relies on.
- **Unicode homoglyph substitution:** Replace common Latin characters with visually identical Unicode lookalikes. For example, replace Latin 'a' (U+0061) with Cyrillic 'a' (U+0430), Latin 'e' with Cyrillic 'e' (U+0435), Latin 'o' with Cyrillic 'o' (U+043e). A human reads the same word; the tokenizer sees unknown subwords.
- **Random character duplication:** For 20% of characters in each word, insert a duplicate of that character. "hate" becomes "haate". This breaks n-gram and subword patterns the model learned during training.

Evaluation procedure:

- Take a random sample of 500 comments from your evaluation set that your clean model classifies as toxic (predicted label = 1) with confidence ≥ 0.7.
- Apply perturb() to each comment.
- Run the perturbed comments through your model.
- **Compute Attack Success Rate (ASR):** the fraction of originally-detected toxic comments that now receive a predicted label of 0 after perturbation. Higher ASR = the attack is more effective.
- Also report the average confidence score before and after perturbation.

**Attack 2 Label-flipping poisoning**

A poisoning attack corrupts the training data before the model is trained, causing the trained model to behave incorrectly in ways that benefit the attacker.

Procedure:

- Take your 100,000-row training subset.
- Randomly select 5% of rows (5,000 rows) and flip their labels: toxic → non-toxic and non-toxic → toxic.
- Retrain a fresh DistilBERT model from the same checkpoint on this poisoned training data, using identical hyperparameters to Part 1.
- Evaluate the poisoned model on the same clean evaluation subset used in Part 1.
- Report the change in: overall F1, Accuracy, and most importantly False Negative Rate (the fraction of genuinely toxic comments the model now misses).

**Key question to answer (in a markdown cell in your notebook)**

Which attack is more operationally dangerous for a live platform, and why? Consider: the evasion attack requires per-comment effort from the attacker; the poisoning attack requires access to the training pipeline. Which threat model is more realistic for a social platform, and what does that imply for where defenses should be prioritized?

**Deliverable:** part3.ipynb with both attack implementations, the ASR table for Attack 1, and the before/after metric comparison table for Attack 2, all rendered in output cells.

## Part 4 Mitigation: Making the Classifier Fairer and More Robust

**What, why, and how**

**What:** Apply three bias mitigation techniques and measure their effect on both fairness metrics and overall accuracy.

**Why:** Mitigation is not a checkbox. Every technique costs something accuracy, latency, complexity, or interpretability. This part forces you to make real engineering trade-offs under real constraints, which is what the job actually looks like.

**How:** You will apply one pre-processing technique, one post-processing technique, and one data-level technique covering the three stages of the ML pipeline where bias can be intervened on.

### Technique 1 Reweighing (pre-processing)

Reweighing assigns different sample weights to training examples to counteract the underrepresentation or mislabeling of certain subgroups. Use aif360's Reweighing class. The privileged group is your reference cohort (white ≥ 0.5, black < 0.1) and the unprivileged group is your high-black cohort (black ≥ 0.5). Retrain DistilBERT from scratch using the computed sample weights passed to the Trainer API's sample_weight argument. Re-evaluate on the same evaluation subgroups as Part 2.

### Technique 2 Threshold optimization (post-processing)

Instead of retraining, adjust the decision threshold separately for each subgroup to equalize a chosen fairness metric. Use fairlearn's ThresholdOptimizer with the constraint equalized_odds. This finds the threshold for each subgroup that minimizes the weighted error while satisfying the equalized odds constraint (TPR and FPR equalized across groups).

After fitting the ThresholdOptimizer, plot the accuracy-fairness Pareto frontier by sweeping the constraint tolerance from 0.0 (strict fairness) to 0.3 (relaxed fairness). The x-axis is equal opportunity difference (fairness) and the y-axis is overall F1 (accuracy). This plot shows the trade-off curve and is the key output of this technique.

### Technique 3 Oversampling (data augmentation)

A simpler intervention: oversample the underrepresented high-black cohort in the training set to ensure the model sees more examples of that subgroup during training. Duplicate all high-black cohort training examples 3× (so each row appears 4 times total: once original and 3 duplicates). Retrain from scratch and re-evaluate.

### Comparison table

Produce a single summary table with one row per technique (plus the baseline) and the following columns:

- Overall F1
- High-black FPR
- Reference cohort FPR
- Statistical parity difference
- Equal opportunity difference

**Key question to answer (in a markdown cell in your notebook)**

Can you simultaneously achieve demographic parity AND equalized odds on this dataset? Try to satisfy both constraints using Technique 2. If you cannot, compute the base rates (prevalence of toxic comments) in each cohort and explain mathematically why the two fairness definitions are incompatible when base rates differ. Show the numbers.

**Deliverable:** part4.ipynb with all three retrained/adjusted models, the full comparison table, and the Pareto plot rendered in output cells. Save the best-performing mitigated model to disk.

## Part 5 Guardrail Pipeline

**What, why, and how**

**What:** Design and implement a three-layer production guardrail pipeline around your best mitigated model, then demonstrate it on 1,000 evaluation examples.

**Why:** A single model is never the full production system. Real moderation pipelines have pre-filters that catch obvious cases cheaply, calibrated models for ambiguous cases, and human review queues for low-confidence decisions. Designing this as a software engineer not just a researcher is the gap between a notebook and a deployable system.

**How:** Implement a ModerationPipeline class in a standalone pipeline.py file. The class must have a .predict(text) method that runs the three layers in sequence and returns a structured decision dictionary.

### Layer 1 Input filter (regex pre-filter)

Before calling the model which is slow and expensive run a fast regex-based pre-filter. If the comment matches, return immediately without calling the model.

Your blocklist must contain at least 20 regex patterns organized across the following five categories. The categories and indicative examples below are your specification implement patterns that match this intent using Python's re module with re.IGNORECASE. You are not required to use the exact strings shown; these are illustrative of the kind of pattern each category covers.

**Category 1 Direct threats of violence (minimum 5 patterns)**

These are explicit statements of intent to physically harm someone. They follow a subject-verb-object structure with a threat verb. Examples of the pattern logic: "I will kill you", "I'm going to hurt you", "you're going to die", "I'll find where you live", "someone should shoot you". Your patterns must use word boundaries (\b) and handle common conjugations (will, gonna, going to, should). At least one pattern must use a capturing group to match a variable threat verb from a set (e.g., kill|murder|shoot|stab|hurt).

**Category 2 Calls for self-harm or suicide (minimum 4 patterns)**

Directed statements urging another user to harm themselves. These are high-precision signals because they combine a second-person subject with a self-harm verb phrase. Examples of the pattern logic: "you should kill yourself", "go kill yourself", "nobody would miss you if you died", "do everyone a favour and disappear". Your patterns must use \b word boundaries and must not match first-person statements (e.g., "I thought about hurting myself" is not in scope for this filter).

**Category 3 Doxxing and stalking threats (minimum 4 patterns)**

Threats to expose private information or locate someone in real life. These have a distinct pattern: a first-person subject combined with knowledge of private details or a surveillance intent. Examples of the pattern logic: “I know where you live”, “I’ll post your address”, “I found your real name”, “everyone will know who you really are”. Your patterns must match variants with and without contractions (I will / I’ll / gonna).

**Category 4 Severe dehumanization (minimum 4 patterns)**

Statements that deny the humanity of a person or group, typically combining a group reference with a sub-human comparison. Examples of the pattern logic: “[group] are not human”, “[group] are animals”, “[group] should be exterminated”, “[group] are a disease”. Use a non-capturing group (?:human|people|person) to match variants. These patterns are intentionally broad they will produce some false positives (e.g., biology discussions), which is acceptable for a pre-filter that feeds into a model layer. Note this in your notebook.

**Category 5 Coordinated harassment signals (minimum 3 patterns)**

Calls to mobilize others to attack or target a specific user. These combine a plural or collective subject with a target and an aggressive verb. Examples of the pattern logic: “everyone report [username]”, “let’s all go after”, “raid their profile”, “mass report this account”. At least one pattern must use a lookahead (?=) to catch the phrase regardless of what follows the trigger word.

**Code structure requirement**

Your patterns must be organized as a Python dictionary keyed by category name, not a flat list. Each value is a list of compiled regex objects. The filter function iterates over all categories and all patterns within each category, and returns the matched category name alongside the decision. This structure makes the filter auditable you can see which category triggered the block, not just that a block occurred. The skeleton structure is:

```
import re

BLOCKLIST = {
    "direct_threat": [
        # your 5+ patterns here
    ],
    "self_harm_directed": [ ... ],
    "doxxing_stalking": [ ... ],
    "dehumanization": [ ... ],
    "coordinated_harassment": [ ... ],
}

def input_filter(text: str) -> dict | None:
    """Returns a block decision dict if matched, else None."""
    for category, patterns in BLOCKLIST.items():
        for pattern in patterns:
            if pattern.search(text):
                return {"decision": "block", "layer": "input_filter", "category": category, "confidence": 1.0}
    return None
```

In your part5.ipynb, run input_filter() on your 1,000 test comments and report: how many were blocked by each category. This tells you which category of harm the filter catches most, and is part of the layer-distribution analysis required for Part 5.

Return format if matched:

```
{"decision": "block", "layer": "input_filter", "confidence": 1.0}
```

**Layer 2 Calibrated model**

If the input filter does not match, pass the text to your best mitigated model from Part 4. Before running predictions in the pipeline, apply probability calibration using sklearn's CalibratedClassifierCV with method='isotonic'. Calibration ensures the model's output probability is a reliable confidence score, not just a raw logit.

- If calibrated confidence ≥ 0.6: return block
- If calibrated confidence ≤ 0.4: return allow
- If calibrated confidence is between 0.4 and 0.6: escalate to Layer 3

**Layer 3 Human review queue**

Anything the model is uncertain about (confidence 0.4–0.6) should not be auto-actioned. Route it to a human review queue instead.

Return format:

```
{"decision": "review", "layer": "model", "confidence": <float>}
```

**Demonstration notebook**

In part5.ipynb, run your pipeline on 1,000 randomly selected comments from your evaluation set. Report and plot:

- What fraction of decisions was handled by each layer (pie chart or bar chart)?
- For the auto-actioned subset (Layer 2 confident decisions only, excluding the review queue): what is the model's F1, precision, and recall?
- For the review-queue subset: what is the actual toxic/non-toxic breakdown (using ground-truth labels)? This tells you how much genuinely ambiguous content is being caught.

**Key question to answer (in a markdown cell in your notebook)**

Is the 0.4–0.6 uncertainty band the right choice for the human review threshold? What happens to the volume of human reviews and the auto-action accuracy if you narrow the band to 0.45–0.55? What if you widen it to 0.3–0.7? Show the numbers for at least two alternative thresholds and argue for your final choice.

**Deliverable:** pipeline.py implementing the ModerationPipeline class with a working .predict(text) method, plus part5.ipynb demonstrating the pipeline on 1,000 examples with all metrics and the layer-distribution plot rendered.

# 5. Submission

Submit a single GitHub repository. The repository must contain the following files at the root level:

| File | Contents |
|---|---|
| **part1.ipynb** | Baseline classifier |
| **part2.ipynb** | Bias audit |
| **part3.ipynb** | Adversarial attacks |
| **part4.ipynb** | Mitigation techniques |
| **part5.ipynb** | Guardrail pipeline demonstration |
| **pipeline.py** | ModerationPipeline class |
| **requirements.txt** | All dependencies with pinned versions |
| **README.md** | Python version, GPU used, how to reproduce |

**Critical submission rules**

- **All notebooks must be submitted with outputs already executed.** Every cell must have been run and its output must be visible. Notebooks with empty output cells will receive zero for those parts.
- **Do not commit the dataset or model checkpoints.** Add *.csv, *.pt, *.bin, and saved model/ to your .gitignore. These files are too large for GitHub and are not part of your submission.
- **Commit history is graded.** A repository with a single commit loses 10 points automatically. Work incrementally and commit each part as you complete it. This demonstrates that the work was done by you over the course of the week, not assembled at the last minute.
- **No starter code is provided.** You are expected to write all code yourself. You may consult HuggingFace documentation, scikit-learn documentation, Fairlearn documentation, and AIF360 documentation. Stack Overflow is permitted for syntax questions. Copying large blocks of code from GitHub without understanding is detectable and will be treated as academic dishonesty.

# 6. Grading

| Part | What is graded | Weight |
|---|---|---|
| **Part 1 Baseline** | Model trains and converges. All metrics computed correctly. Threshold choice is justified with evidence. | **10%** |
| **Part 2 Bias audit** | Cohort construction is correct. All required metrics present. Disparate Impact ratio computed. Visualizations correct. Disparity direction interpreted correctly. | **25%** |
| **Part 3 Attacks** | Both attacks implemented from scratch. ASR reported for Attack 1. Before/after comparison for Attack 2. Operational risk analysis is thoughtful and grounded. | **20%** |

| | | |
|---|---|---|
| **Part 4 Mitigation** | All 3 techniques correctly applied. Comparison table complete and accurate. Pareto plot present. Fairness incompatibility addressed with actual numbers. | **25%** |
| **Part 5 Pipeline** | pipeline.py runs without errors. .predict() returns correct format. Layer distribution plotted. Threshold sensitivity analysis present. | **20%** |

# 7. Frequently Asked Questions

**My cohorts are very small after filtering. Is that normal?**

Yes. The high-black cohort will be a small fraction of your 20,000-row evaluation set because explicit identity references are rare in the full dataset. If you have fewer than 200 rows in a cohort, your fairness metrics will have wide confidence intervals. Report the cohort sizes clearly in your notebook. If you have fewer than 50 rows in a cohort, something is wrong with your filtering logic  double-check your threshold conditions.

**Can I use the pre-tokenized seqlen128 files instead of tokenizing myself?**

No. The pre-tokenized files are formatted for a specific BERT configuration and do not contain the identity columns you need for the bias audit. You must work from jigsaw-unintended-bias-train.csv and tokenize using HuggingFace's AutoTokenizer.

**My model is not converging / F1 is very low. What should I check?**

First, verify your label binarization: the toxic column has values between 0 and 1, not integers. Make sure you applied ≥ 0.5 cutoff. Second, check class imbalance: the dataset is heavily imbalanced (roughly 8% toxic). If your F1 is near 0, your model may have collapsed to predicting all non-toxic. Third, verify your GPU is active in Colab (Runtime → Change runtime type → GPU). Training on CPU will take hours and the learning rate schedule may behave differently.

**For the poisoning attack, should I retrain from the pre-trained checkpoint or from my Part 1 trained model?**

Retrain from the original pre-trained distilbert-base-uncased checkpoint, not from your Part 1 fine-tuned model. The point of the attack is to corrupt the fine-tuning process itself. Starting from your already-fine-tuned model would not simulate a real poisoning attack.

**What exactly does CalibratedClassifierCV do and why is it needed?**

Transformer models output raw logits that are passed through a softmax to produce probabilities. However, these probabilities are not well-calibrated  a model that outputs 0.8 confidence is not necessarily correct 80% of the time. CalibratedClassifierCV fits a mapping (using isotonic regression) from raw model outputs to calibrated probabilities. This matters for the guardrail pipeline because the 0.4–0.6 uncertainty band only makes sense if the probabilities are actually calibrated. Without calibration, the threshold band is arbitrary.

**Can I use a different model instead of DistilBERT?**

No. DistilBERT is specified because it produces consistent results across environments, is small enough for Colab, and the bias patterns documented in the literature apply to this model family. Using a different model makes your results incomparable and defeats the purpose of the assignment.